

Cell 8 — Skip-gram Word2Vec + Sequence Models (Notebook cell index: 16)

Trains Word2Vec (skip-gram), builds frozen embedding matrix, tokenizes/pads text, trains six RNN architectures with grid search per preprocessing.

Granular line-by-line breakdown

Globals

```
MAX_WORDS = 30000
MAX_LEN = 60
EMBED_DIM = 100
```

- **MAX_WORDS** — Maximum tokenizer vocabulary size (matches Embedding rows cap).
 - **MAX_LEN** — Timesteps per padded sequence (truncate/pad length).
 - **EMBED_DIM** — Must match `Word2Vec(vector_size=...)`.
-

build_embedding_matrix

```
def build_embedding_matrix(word_index, w2v, max_words=MAX_WORDS, embed_dim
```

- **word_index** — Keras `tokenizer.word_index`: dict `word` → `integer index` (starts at 1).
- **w2v** — Trained `gensim.models.Word2Vec` model.

```
    vocab_size = min(max_words, len(word_index) + 1)
```

- `len(word_index) + 1` — Indices 1..V plus padding row 0 → **vocab_size** rows for Embedding layer.
- `min(max_words, ...)` — Caps rows at **MAX_WORDS** if tokenizer learned more words.

```
    emb_matrix = np.random.normal(scale=0.6, size=(vocab_size, embed_dim))
```

- `np.random.normal` — Gaussian initialization for rows not filled from Word2Vec.
- `size=(vocab_size, embed_dim)` — 2D weight matrix.

```
    emb_matrix[0] = np.zeros(embed_dim)
```

- **Row index 0** — Reserved for **padding token** in padded sequences; zeros avoid adding noise.

```
    for word, idx in word_index.items():
        if idx >= vocab_size:
            continue
        if word in w2v.wv:
            emb_matrix[idx] = w2v.wv[word]
```

- `word_index.items()` — Loop each `(word, idx)` pair.
- `if idx >= vocab_size` — Skip indices outside truncated embedding table.
- `w2v.wv` — Gensim **KeyedVectors**: `w2v.wv[word]` returns that word's vector if trained.
- `emb_matrix[idx] = ...` — Copy pretrained vector into row `idx` aligned with Keras tokenizer.

```
    return emb_matrix
```

tokenize_pad

```
def tokenize_pad(X_train_text, X_val_text, X_test_text):
```

- Three equal-length splits of **strings** (already preprocessed).

tokenizer = Tokenizer(num_words=MAX_WORDS, oov_token='<OOV>')
- num_words=MAX_WORDS — Only top frequencies get indices **below** cap (implementation detail: indices still exist but num_words limits internal ranking usage).
- oov_token='<OOV>' — Reserve token for unknown/out-of-vocabulary words.

tokenizer.fit_on_texts(X_train_text)
- Builds word frequency ranking **from training strings only** (no validation/test leakage).

X_train_seq = tokenizer.texts_to_sequences(X_train_text)
- texts_to_sequences — Each document → **list of integer token IDs**.

X_train_pad = pad_sequences(X_train_seq, maxlen=MAX_LEN, padding='post'
- pad_sequences — Converts variable-length lists to fixed maxlen arrays.
- padding='post' — Adds zeros **after** real tokens (end padding).
- truncating='post' — If longer than MAX_LEN, cuts off **end** (keeps beginning). Same for val/test.

return tokenizer, X_train_pad, X_val_pad, X_test_pad

build_seq_model

```
def build_seq_model(model_type, vocab_size, emb_matrix, units=64, dropout
```

- model_type — String selecting RNN class.
- vocab_size — Embedding input_dim (rows count).
- emb_matrix — Pretrained weights array.

model = Sequential()
model.add(Embedding(
 input_dim=vocab_size,
 output_dim=emb_matrix.shape[1],
 weights=[emb_matrix],
 trainable=False
))
- Embedding — Maps integer sequence → sequence of vectors.
- output_dim=emb_matrix.shape[1] — Embedding width matches Word2Vec dimension.
- weights=[emb_matrix] — Initialize from numpy matrix (list wrapper required by Keras).
- trainable=False — **Freeze** embeddings (skip-gram vectors fixed).

if model_type == 'SimpleRNN':
 model.add(SimpleRNN(units))
elif model_type == 'GRU':
 model.add(GRU(units))
...
elif model_type == 'BiLSTM':
 model.add(Bidirectional(LSTM(units)))
else:
 raise ValueError('Unknown model type')
- elif chain — Picks recurrent layer type.

- **Bidirectional(...)** — Runs forward + backward RNN; **units** is per direction internal state size (implementation-dependent behavior).

```
model.add(Dropout(dropout_rate))
model.add(Dense(num_classes, activation='softmax'))
```

- **Dropout** — After RNN output vector (last timestep output for unidirectional; concatenated for bidirectional).
- **Dense(..., softmax)** — Class probabilities (**num_classes** from outer notebook scope).

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=lr),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
return model
```

Model list and grid

```
seq_model_names = ['SimpleRNN', 'GRU', 'LSTM', 'BiSimpleRNN', 'BiGRU', 'Bi
seq_grid = {
    'units': [64, 96],
    'dropout_rate': [0.4, 0.5],
    'lr': [1e-3],
    'batch_size': [64],
    'epochs': [50]
}
```

- **seq_model_names** — Strings passed as **model_type**.
- **seq_grid** — Swept by **ParameterGrid**.

Main outer loop (same as TF-IDF cell)

```
for prep_name, prep_fn in PREPROCESSORS.items():
    ...
    X_train_text, X_val_text, X_test_text = get_preprocessed_splits(prepare_fn,
    train_sent_tokens = [txt.split() for txt in X_train_text]
```

- **List comprehension** — Each headline string → **list of word strings** for Word2Vec (gensim expects sentence-like lists).

Word2Vec training

```
w2v = Word2Vec(
    sentences=train_sent_tokens,
    vector_size=EMBED_DIM,
    window=5,
    min_count=2,
    sg=1,
    workers=4,
    epochs=50,
    seed=SEED
)
```

- **sentences** — Corpus for training (training split only).
- **vector_size** — Dimension of each word vector (**must match EMBED_DIM**).
- **window=5** — Context window half-width.
- **min_count=2** — Ignore words appearing once.
- **sg=1** — **Skip-gram** mode (0 would be CBOW).

- `workers=4` — Parallel threads.
- `epochs=50` — Training epochs over corpus.
- `seed=SEED` — Reproducibility hook.

Tokenize + embedding matrix

```
tokenizer, X_train_pad, X_val_pad, X_test_pad = tokenize_pad(X_train_t
```

- Unpacks padded integer matrices.

```
vocab_size = min(MAX_WORDS, len(tokenizer.word_index) + 1)
```

- `tokenizer.word_index` — Dict size drives vocabulary; +1 accounts for padding index 0.

```
emb_matrix = build_embedding_matrix(tokenizer.word_index, w2v)
```

Per architecture inner loops

```
for model_name in seq_model_names:
    best_f1 = -1
    best_model = None
    early_stop = EarlyStopping(monitor='val_loss', patience=2, restore
```

```
    for cfg in ParameterGrid(seq_grid):
        model = build_seq_model(
            model_type=model_name,
            vocab_size=vocab_size,
            emb_matrix=emb_matrix,
            units=cfg['units'],
            dropout_rate=cfg['dropout_rate'],
            lr=cfg['lr']
        )
```

- `cfg` — One combination from `seq_grid`.

```
        model.fit(
            X_train_pad, y_train_oh,
            validation_data=(X_val_pad, y_val_oh),
            epochs=cfg['epochs'],
            batch_size=cfg['batch_size'],
            verbose=1,
            callbacks=[early_stop]
        )
```

- `X_train_pad` — Shape (n_train, MAX_LEN) integers.

```
        val_pred = np.argmax(model.predict(X_val_pad, verbose=0), axis
        val_f1 = f1_score(y_val, val_pred, average='macro')
        add_tuning_log(prepare_name, 'Skip-gram', model_name, cfg, val_f1
```

- `'Skip-gram'` — Representation label for logs.

```
        if val_f1 > best_f1:
            best_f1 = val_f1
            best_model = model
```

```
        test_pred = np.argmax(best_model.predict(X_test_pad, verbose=0), a
        evaluate_and_store(y_test, test_pred, prepare_name, 'Skip-gram', mode
```

- After exploring `cfg`, evaluate **best** model on test for this `model_name`.

Mechanism of the whole cell

Preprocess strings → list of tokens → train skip-gram Word2Vec → Keras sequences + padding → embedding lookup table → try six RNN types × hyperparameter grid → log val, pick best, test.

Role and importance in the notebook

Implements **distributional semantics** + **recurrent networks** track and compares it with TF-IDF experiments under the same preprocessing keys.